



DRAW IT or LOSE IT!

Software Design
Version 1.0

Table of Contents

Software Design	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	Error! Bookmark not defined.
Domain Model	3
Evaluation	4
Recommendations	6

Document Revision History

Version	Date	Author	Comments
1.0	05/23/2026	Adalberto Flores	Refine the design to use a shared Entity base class and singleton GameService.

Executive Summary

The Gaming Room wants to move **Draw It or Lose It** from single Android app to a web-based system that works on multiple platforms. My design focuses on a simple object-oriented model using shared Entity, Game, Team, and Player classes so the game data is easy to manage and reuse. I use a singleton GameService to control all games, teams, and players from one place, which keeps the logic consistent as the app grows. This approach gives the client a clean base to expand the game across different operating systems while keeping the code organized and easier to maintain.

Requirements

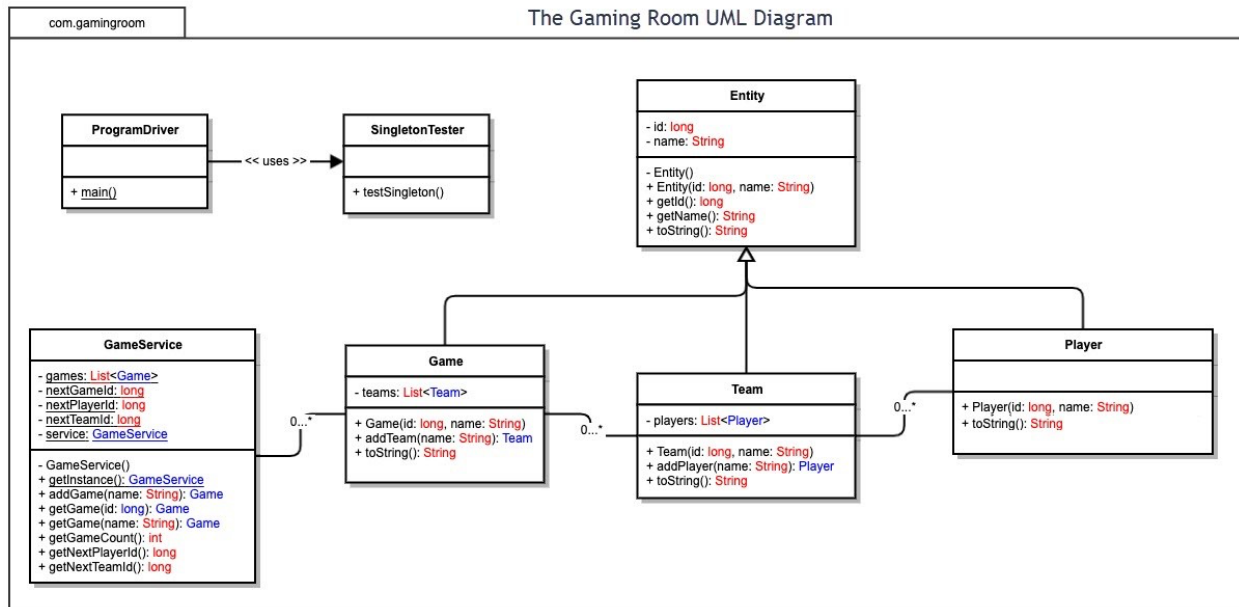
Key requirements for the game are straightforward. The game must support one or more teams, and each team needs multiple players. Game and team names must be unique so users can see if a name is already taken. The system should also only keep one instance of the game service in memory at a time, using unique IDs for each game, team, and player to keep everything organized across platforms.

Design Constraints

A few designs constrain shape how I built this version of the game. The game must run in a web based, distributed environment, so the design needs to be simple enough to work across different operating systems without depending on one specific platform. Because only one instance of the game service can exist in memory, I used a singleton GameService to control all games, teams, and players from a central place. Game, team, and player names also need to stay unique, which means the code must always check existing collections with iterators before creating anything new. The constraints push the design toward a clean object-oriented model that is easier to scale and host on multiple platforms later.

Domain Model

The domain model is built around the Entity base class, which stores the common id and name fields shared by Game, Team and Player. This shows inheritance because those three classes reuse the same core data instead of duplicating it in each class. The model also shows encapsulation by keeping the data inside classes and accessing it through constructors and getter methods. GameService manages the collections of games, teams, and players and uses the singleton pattern so only one service instance exists in memory at a time. Iterator-based searches in the add and get methods help enforce unique names for games, teams and players, which supports the client's software requirement in an efficient and organized way.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac clients just need the web app tested in Safari and other macOS browser, so the layout and controls	Linux clients mainly add a testing step in Firefox and chrome on a couple of common distros, since the	Windows clients require testing in Edge and other Windows browsers to make sure the UI, input, and performance	Mobile clients need a responsive design that feels good on phones and tablets, so the main cost is extra work to handle touch input, smaller

	work the same as on other desktops.	same standards-based HTML, CSS, and JavaScript should work there with no extra code.	match what players see on the other desktop platforms.	screens, and different mobile browser quirks.
Client Side	Make sure the responsive web app works cleanly in Safari and other macOS browsers so players get the same desktop experience as everywhere else.	Test the same HTML, CSS and JavaScript in Firefox and Chrome on a couple of common Linux distros to confirm the web client behaves the same there without extra code.	Check the web app in Edge and other Windows browsers so the UI, input, and performance match what users see on Mac and Linux.	Use a responsive design that feels good on phones and tablets, with extra effort on touch input and smaller screens while using the same web base.
Development Tools	Use web tech like HTML, CSS, JavaScript, and a backend language such as Java or Node with tools like VS Code or IntelliJ, which all run fine on macOS with only optional IDE license costs.	Use the same web stack with open-source tools like VS Code, Java, Node, and typical Linux terminals, which keeps licensing costs low and matches what is usually run in production servers.	Use HTML, CSS, JavaScript, and a backend stack that can run on Windows along with IDEs like Visual Studio or IntelliJ, which may add some licensing costs, but many tools also have free versions.	Rely on standard web technologies so the same code runs in mobile browser, using desktops IDEs and browser dev tools for debugging instead of any heavy native mobile toolchains.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform:** To help The Gaming Room expand Draw It or Lose It to different devices, I recommend using a Linux-based server platform such as Ubuntu Server or Red Hat Enterprise Linux, hosted in a cloud environment like AWS or Microsoft Azure. Linux is already a standard choice for web servers because it is stable, open source, and easier to scale as the number of players grows. Running the game as a centralized web service on Linux means players on Mac, Windows, Linux, and mobile only need a normal web browser to connect. This platform also gives the Java-based backend and the singleton GameService a reliable place to run so it can handle more concurrent users over time without locking the game to one client platform.
2. **Operating Systems Architectures:** The recommended Linux server uses a monolithic kernel architecture that keeps core OS services efficient while still protecting the application. The low-level functions such as CPU scheduling, memory management, and device control run in kernel space, while the Draw It or Lose It application and GameService run in user space. That separation matters because a bug in the game code is less likely to crash the whole server. Linux also handles multithreading and multiprocessing well, which helps the game manage multiple active sessions and player connections at the same time. In practice this supports a simple multi-tier web setup where Linux hosts the web server, application logic, and database connections in a way that is predictable and easier to scale later.
3. **Storage Management:** For storage management, a relational database such as PostgreSQL or MySQL running on the Linux server is a good fit for Draw It or Lose It. The game relies on structured data like users, teams, games, and scores that need to stay consistent as more people play. On the operating system side, Linux file systems like ext4 or XFS organize data in a standard directory tree and use inodes and journaling to track files and recover more safely after a crash. The database can store persistent data such as player profiles, team setups, and historical game results, while short lived game state can stay in memory or in temporary files when needed. This mix of a relational database and a journaling file system gives the game reliable storage now and room to scale later.
4. **Memory Management:** The Linux operating platform uses virtual memory with paging, which helps it manage memory for the Draw It or Lose It server even when load increases. Virtual memory lets the application see a continuous address space while the OS maps pages to physical RAM under the hood. If RAM fills up during busy periods, Linux can move less active pages to swap space so the system keeps running instead of failing outright. Since the game is written in

Java, the Java Virtual machine also plays a big role in memory management. The JVM garbage collector automatically frees memory from Entity, Team, Player, and Game objects that are no longer in use, which reduces memory leaks and helps the GameService stay responsive over time.

5. **Distributed Systems and Networks:** To support players on Mac, Windows, Linux, and mobile, the game should follow a client server distributed model where most of the logic lives on the central Linux Server. The server exposes RESTful web APIs, and clients send HTTP or HTTPS requests over the TCP/IP stack to submit moves and get updates. This keeps the rules and game state in one place while letting different devices interact with it through a browser. Because distributed systems always depend on the network, the design has to expect latency and dropped connections. The client side should handle slow responses with simple loading states and timeouts, and the server should save game state often enough that player can reconnect and continue their current game instead of starting over.
6. **Security:** Security needs to be in place at the network, server, and application levels to protect user information across all platforms. All traffic between the clients and the Linux server should use HTTPS with TLS so that credentials and other sensitive data are encrypted in transit. On the server, Linux can use standard file permissions, separate user accounts, and tools like ufw or iptables to limit which ports and services are exposed. The game server process should run under a dedicated service account that only has access to the files and resources it actually needs. At the application level, passwords should be stored using strong hashing algorithms such as bcrypt, and user sessions should rely on signed authentication tokens. Adding basic logging for failed logins and usual activity will make it easier to spot potential security issues and respond quickly.